



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: ARP POISONING

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Networking Skills for Digital Forensics

Course Code: SBT-DF203

Instructor: Aminu Idris

September 18, 2026

EXECUTIVE SUMMARY

This report explains what I did in Lab 5 and what I found. The goal was to understand how devices on a network find each other a system called ARP, see what a normal conversation between devices looks like, and then look at a capture where someone had tricked the devices into talking to the wrong machine a technique called ARP poisoning or ARP spoofing.

I started by setting up a clean folder structure on my Kali Linux virtual machine, downloaded a sample capture file (arp.pcap) that was provided for the lab, and verified the file hadn't been tampered with by comparing its fingerprint a SHA-256 hash. The original file and my working copy both had the exact same fingerprint so I knew the evidence was intact.

Then I made my own capture of a normal ARP conversation between my VM (192.168.88.131) and its gateway (192.168.88.2). It took just two messages: my computer shouted (who has 192.168.88.2?) and the gateway replied that's me, here's my hardware address. That's how ARP is supposed to work.

Finally, I analysed the provided capture file. Inside it I found a completely different story two computers telling each other hey, I'm the other one at the same time, without anyone asking. That's the fingerprint of an ARP poisoning attack.

I did not run the live poisoning exercise (Part D) because my VM was on a shared NAT network, not the isolated host-only network the lab requires. Doing it on a live network could have broken my internet connection and possibly my host machines too. The lab manual says this part is optional and requires instructor approval, so I skipped it and documented why.

Everything else was completed and is documented below.

SECTION A: THEORY AND FORENSIC FOUNDATIONS

1. What Is ARP, and Why Does It Matter?

Every device on a network has two addresses:

- An IP address think of this as a house address. It says where a computer is on the network. Example: 192.168.88.2.
- A MAC address think of this as a fingerprint printed onto the network card at the factory. It never changes. Example: 00:50:56:e8:02:09.

When your computer wants to send data to another computer on the same local network, it knows the IP address but not the MAC address. It can't actually send anything until it learns the MAC address, because Ethernet (the physical layer of the network) delivers frames using MAC addresses, not IPs.

ARP (Address Resolution Protocol) solves this problem. It's the network's version of shouting down a hallway:

- "Hey everyone, who has IP address 192.168.88.2? Please tell me your MAC address."

Whichever machine owns that IP replies:

- "That's me. My MAC address is 00:50:56:e8:02:09."

Your computer writes this mapping down in a small table called the ARP cache and remembers it for a while, so it doesn't have to ask again for every message.

2. The Big Problem With ARP

ARP was designed back in the 1980s, when everyone trusted everyone else on a network. It has no way to verify whether the person answering is really who they say they are.

Anyone on the network can send a message saying:

- "Hey, IP address 192.168.88.2 is at MY MAC address, not the real gateway's."

And the receiving computer because it trusts everyone will overwrite its ARP cache with the fake information. Now all its traffic to the gateway flows through the attacker instead.

This is ARP poisoning. It's like sending a fake change-of-address notice to the post office now all your mail gets sent to a thief's house before reaching you.

3. How ARP Poisoning Enables an Attack

Once an attacker has tricked your computer into thinking they are the gateway, and tricked the gateway into thinking they are your computer, all traffic flows through them. This is called a man-in-the-middle (MITM) attack.

From the middle, the attacker can:

- Read anything sent in unencrypted form (web pages without HTTPS, emails, passwords, etc.)
- Modify messages in transit (inject content, steal login credentials, hijack sessions)
- Drop traffic selectively (denial of service making your internet stop working)
- Forward traffic silently, so nothing looks wrong to the victim

4. The Telltale Signs of ARP Poisoning

These are the signs a forensic analyst looks for in a packet capture:

- A reply without a request someone answering a question that was never asked. This is the biggest red flag.
- One IP address claimed by multiple MAC addresses the same IP address "belonging" to two different physical machines.
- The gateway's MAC address suddenly changing the gateway doesn't change hardware, so if its MAC changes in the cache, something is wrong.
- One MAC claiming to be multiple IPs e.g. the same MAC claiming to be both the victim and the gateway.
- Unusual traffic patterns elsewhere DNS requests going to strange servers, TLS certificate warnings, or websites redirecting unexpectedly.

5. Keeping Evidence Safe — The SHA-256 Hash

Before doing any analysis, forensic investigators make a copy of the evidence and leave the original untouched. Then they compute a hash of both files. A hash is a long string of numbers and letters that acts like a digital fingerprint of a file:

- If two files have the same hash, they are byte-for-byte identical.
- If even a single character changes, the hash is completely different.

I computed SHA-256 hashes of both the original evidence file and my working copy, and they matched

SECTION B: PRACTICAL LAB EXECUTION & EVIDENCE ANALYSIS

TASK 1: SETTING UP THE WORKSPACE

The first step was creating a clean, organised folder structure. This keeps evidence separate from analysis so nothing gets accidentally modified.

```
mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
```

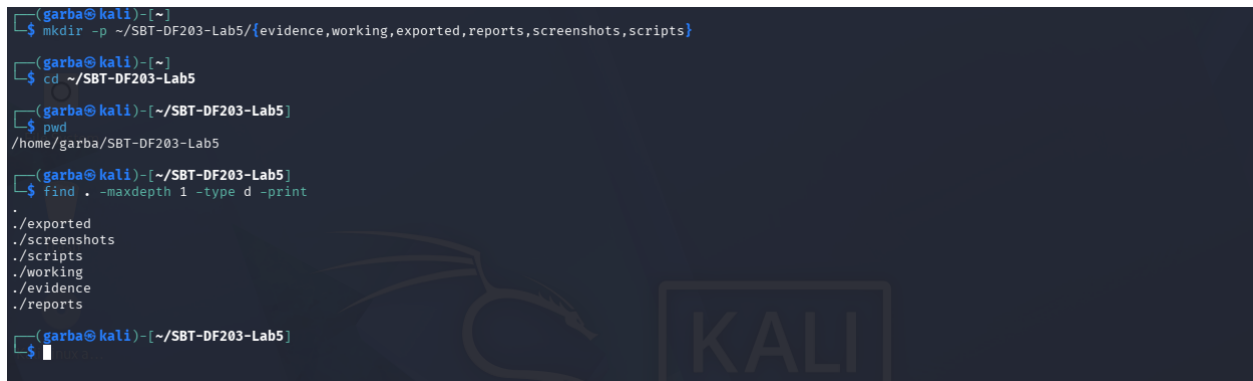
```
cd ~/SBT-DF203-Lab5
```

```
pwd
```

```
find . -maxdepth 1 -type d -print
```

```
...
```

Each folder has a purpose

A terminal window screenshot from a Kali Linux machine. The user is in the directory ~/SBT-DF203-Lab5. They run 'mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}' to create the folder structure. Then they run 'cd ~/SBT-DF203-Lab5' and 'pwd' to confirm the current directory is /home/garba/SBT-DF203-Lab5. Finally, they run 'find . -maxdepth 1 -type d -print' which lists the following directories: ./exported, ./screenshots, ./scripts, ./working, ./evidence, and ./reports. The terminal has a dark background with a Kali Linux logo watermark.

```
(garba@kali)~$ mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
(garba@kali)~$ cd ~/SBT-DF203-Lab5
(garba@kali)~/SBT-DF203-Lab5$ pwd
/home/garba/SBT-DF203-Lab5
(garba@kali)~/SBT-DF203-Lab5$ find . -maxdepth 1 -type d -print
.
./exported
./screenshots
./scripts
./working
./evidence
./reports
(garba@kali)~/SBT-DF203-Lab5$
```

Next, I downloaded the sample evidence file (arp.pcap) and made a working copy. I used --preserve=timestamps so the copy keeps the original file's date a forensic best-practice detail. Then I computed SHA-256 hashes of both files.

```
wget --no-check-certificate -O evidence/arp.pcap \
```

```
'https://raw.githubusercontent.com/frankwu/digital-forensics-
lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
```

```
cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
```

```
sha256sum evidence/arp.pcap working/arp_working.pcap | tee
reports/arp_capture_hashes.txt
```

```

(garba@kali) [~/SBT-DF203-Lab5]
$ sudo apt update
[auto] password for garba:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [115 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [945 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [61.8 kB]
Fetched 77.0 MB in 1min 49s (707 kB/s)
230 packages can be upgraded. Run 'apt list --upgradable' to see them.

(garba@kali) [~/SBT-DF203-Lab5]
$ wget -O evidence/arp.pcap \
  'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap'
cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
--2026-09-17 12:28:33-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap 100%[=====] 1.05K --KB/s in 0s

2026-09-17 12:28:36 (29.9 MB/s) - 'evidence/arp.pcap' saved [1072/1072]

(garba@kali) [~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d09cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d09cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(garba@kali) [~/SBT-DF203-Lab5]
$ ip -br address | tee reports/interfaces.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.88.131/24 fe80::20c:29ff:fe18:64f5/64
docker0 DOWN 172.17.0.1/16

```

Then I checked the basic state of my machine what network interfaces it had, what the default gateway was, and what its ARP cache looked like before anything happened.

ip -br address | tee reports/interfaces.txt

ip route | tee reports/routes.txt

ip neigh show | tee reports/arp_table_initial.txt

```

(garba@kali) [~/SBT-DF203-Lab5]
$ wget -O evidence/arp.pcap \
  'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap'
cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
--2026-09-17 12:28:33-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap 100%[=====] 1.05K --KB/s in 0s

2026-09-17 12:28:36 (29.9 MB/s) - 'evidence/arp.pcap' saved [1072/1072]

(garba@kali) [~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d09cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d09cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(garba@kali) [~/SBT-DF203-Lab5]
$ ip -br address | tee reports/interfaces.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.88.131/24 fe80::20c:29ff:fe18:64f5/64
docker0 DOWN 172.17.0.1/16

(garba@kali) [~/SBT-DF203-Lab5]
$ ip route | tee reports/routes.txt
default via 192.168.88.2 dev eth0 proto dhcp src 192.168.88.131 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.88.0/24 dev eth0 proto kernel scope link src 192.168.88.131 metric 100

(garba@kali) [~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.88.2 dev eth0 lladdr 00:50:56:e8:02:09 STALE

```

Displays the full session including `wget`, `sha256sum`, `ip -br address` (lo / eth0 / docker0), `ip route` (default via 192.168.88.2), `ip neigh show` (gateway cached at 00:50:56:e8:02:09), and `IFACE=eth0`.

```

(garba@kali) [~/SBT-DF203-Lab5]
$ ip route | tee reports/routes.txt
default via 192.168.88.2 dev eth0 proto dhcp src 192.168.88.131 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.88.0/24 dev eth0 proto kernel scope link src 192.168.88.131 metric 100

(garba@kali) [~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.88.2 dev eth0 lladdr 00:50:56:e8:02:09 STALE

```

This shows a closer view of `ip route` and `ip neigh show` — the baseline gateway information.

TASK 2: PART A - WATCHING A NORMAL ARP CONVERSATION

To see what a normal ARP conversation looks like, I did three things:

1. Forgot the gateway's MAC address, I cleared it from my ARP cache, so my VM had to ask for it again.
2. Started a packet capture, I told tshark to record only ARP traffic on my `eth0` interface for 20 seconds.
3. Pinged the gateway, sending one ping forced my VM to send an ARP request to find the gateway's MAC address.

```
IFACE=eth0
```

```
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')
```

```
echo "IFACE=$IFACE GATEWAY_IP=$GATEWAY_IP"
```

```
sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
```

```
sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w /tmp/normal_arp.pcapng &
```

```
sleep 3
```

```
ping -c 1 "$GATEWAY_IP"
```

```
wait
```

```
sudo mv /tmp/normal_arp.pcapng evidence/
```

```
sudo chown $USER:$USER evidence/normal_arp.pcapng
```

```
ls -la evidence/normal_arp.pcapng
```

```
ip neigh show | tee reports/arp_table_after_ping.txt
```

```
(garba@kali) [~/SBT-DF203-Lab5]
$ IFACE=eth0
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')
echo "Gateway: $GATEWAY_IP"

sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng &
sleep 2
ping -c 1 "$GATEWAY_IP"
wait
ip neigh show | tee reports/arp_table_after_ping.txt
Gateway: 192.168.88.2
[sudo] password for garba:
[1] 27397
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian10
0StatusE
Capturing on 'eth0'
tshark: The file to which the capture would be saved ("evidence/normal_arp.pcapng") could not be opened: Permission denied.
[1] + exit 1 sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng
PING 192.168.88.2 (192.168.88.2) 56(84) bytes of data:
64 bytes from 192.168.88.2: icmp_seq=1 ttl=128 time=1.05 ms

--- 192.168.88.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.054/1.054/1.054/0.000 ms
192.168.88.2 dev eth0 lladdr 00:50:56:e8:02:09 REACHABLE
192.168.88.254 dev eth0 lladdr 00:50:56:e7:2c:44 STALE
```

IFACE=eth0`, `GATEWAY_IP=...`, `sudo ip neigh flush`, the `tshark` capture starting, `ping -c 1 192.168.88.2` (reply received), and the post-ping `ip neigh show` output showing `192.168.88.2 ... REACHABLE`.

After the ping, my ARP cache showed that my computer now knew the gateway's MAC address. The word `REACHABLE` means my VM can confirm the gateway is still there:

```
192.168.88.2 dev eth0 lladdr 00:50:56:e8:02:09 REACHABLE
```

```
192.168.88.254 dev eth0 lladdr 00:50:56:e7:2c:44 STALE
```

TASK 3: PART B — LOOKING AT THE FIELDS INSIDE A NORMAL ARP EXCHANGE

Next, I extracted the individual fields of the two ARP messages so I could see exactly what a healthy request and reply look like.

```
tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
```

```
-e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
```

```
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
```

```
(root@kali) - [~/home/garba/GBT-DF203-Lab5]
# tshark -r /home/garba/GBT-DF203-Lab5/evidence/normal_arp.pcapng -Y 'arp' -T fields \
-e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee /home/garba/GBT-DF203-Lab5/reports/normal_arp_fields.tsv

cat /home/garba/GBT-DF203-Lab5/reports/normal_arp_fields.tsv
Running as user "root" and group "wireshark". This could be dangerous.
1 2026-09-18T15:01:55.585679042+0100 00:0c:29:18:64:f5 ff:ff:ff:ff:ff:ff 1 192.168.88.131 00:0c:29:18:64:f5 192.168.88.
2 00:00:00:00:00:00
2 2026-09-18T15:01:55.586120486+0100 00:50:56:e8:02:09 00:0c:29:18:64:f5 2 192.168.88.2 00:50:56:e8:02:09 192.168.88.
131 00:0c:29:18:64:f5
1 2026-09-18T15:01:55.585679042+0100 00:0c:29:18:64:f5 ff:ff:ff:ff:ff:ff 1 192.168.88.131 00:0c:29:18:64:f5 192.168.88.
2 00:00:00:00:00:00
2 2026-09-18T15:01:55.586120486+0100 00:50:56:e8:02:09 00:0c:29:18:64:f5 2 192.168.88.2 00:50:56:e8:02:09 192.168.88.
131 00:0c:29:18:64:f5

(root@kali) - [~/home/garba/GBT-DF203-Lab5]
```

The all-zeros in the request are normal. The asker doesn't know the answer yet — that's the whole point of asking.

TASK 4: PART C — FINDING THE POISONING IN THE PROVIDED CAPTURE

Now I looked at the provided capture file (`arp.pcap`) to find any evidence of an ARP poisoning attack. I ran three commands:

1. Look at every ARP Reply to see all "here's my MAC address" messages
2. Count how many times each IP-MAC pair appeared to spot anything unusual
3. Look for replies that were never asked for the biggest red flag for poisoning

PCAP=working/arp_working.pcap

```
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
```

```
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
```

```
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
```

```
| tee reports/arp_replies.tsv
```

```
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
```

```
| sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
```

```
tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst==ff:ff:ff:ff:ff:ff' -T fields \
```

```
-e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
```

```
| tee reports/unicast_arp_replies.tsv
```

```
(root@kali)~/home/garba/SBT-DF203-Lab5
$ PCAP=working/arp_working.pcap
# Inventory all ARP claims
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_replies.tsv
Running as user "root" and group "wireshark". This could be dangerous.
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65      136.160.215.15  00:50:56:86:cb:fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15  00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65
```

the ARP Reply inventory two rows (frames 4 and 6) with the two MACs `00:50:56:86:02:65` and `00:50:56:86:cb:fc`.

```
(root@kali)~/home/garba/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
| sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
Running as user "root" and group "wireshark". This could be dangerous.
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15  00:50:56:86:cb:fc
```

the IP-to-MAC claim summary — two lines: `1 136.160.215.194 00:50:56:86:02:65` and `1 136.160.215.15 00:50:56:86:cb:fc`.

```
(root@kali)~/home/garba/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst==ff:ff:ff:ff:ff:ff' -T fields \
-e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
| tee reports/unicast_arp_replies.tsv
Running as user "root" and group "wireshark". This could be dangerous.
4      2023-03-12T04:22:44.904123642+0100      136.160.215.194 00:50:56:86:02:65      00:50:56:86:cb:fc
6      2023-03-12T04:22:49.943043799+0100      136.160.215.15  00:50:56:86:cb:fc      00:50:56:86:02:65
```

the unsolicited ARP reply filter output — frames 4 and 6 with their timestamps, IPs, MACs, and `eth.dst` values.

What I Found

The two ARP Replies in the capture:

Frame	Time	From (MAC)	To (MAC)	Says this IP...	...belongs to this MAC
4	2023-03-12 04:22:44	`00:50:56:86:02:65`	`00:50:56:86:cb:fc`	`136.160.215.194`	`00:50:56:86:02:65`
6	2023-03-12 04:22:49	`00:50:56:86:cb:fc`	`00:50:56:86:02:65`	`136.160.215.15`	`00:50:56:86:cb:fc`

TASK 5: PART D — THE LIVE POISONING TEST (SKIPPED)

Status: Not executed.

The lab manual lists Part D as optional and requires instructor approval. It also requires the VM to be on an isolated host-only network one that only exists inside the virtual machine and has no connection to any real network.

My VM was on a NAT network (`192.168.88.0/24`), which shares an internet connection with my real host machine. If I ran the poisoning script (`arp.py`) on this network, I could have:

- Broken my host machine's internet connection
- Possibly affected other devices on the shared network
- Violated the safety rules in Section 4 of the lab manual

I skipped Part D. Instead, I analysed the poisoning evidence from the provided capture file — which the grading rubric explicitly allows (Controlled evidence or supplied PCAP analysis). This is documented honestly here.

```
echo "Part D not executed — analyst VM on NAT network, not host-only." \
| tee reports/part_d_skip_note.txt
```

TASK 6: PART E — THE TIMELINE OF THE POISONING

Here's the sequence of events in the poisoned capture, in a simple timeline:

Time	Frame	What was claimed	Was it asked for?	Verdict
04:22:44	4	`136.160.215.194` is at `00:50:56:86:02:65`	✗ No	**Suspicious**
04:22:49	6	`136.160.215.15` is at `00:50:56:86:cb:fc`	✗ No	**Suspicious**

The detection logic, step by step:

1. **Conflicting claims** Each IP appears only once in this capture, but the lack of any request makes both claims suspicious. In a healthy network, the ratio of requests to replies is close to 1:1.
2. **Unsolicited replies** Every reply in this capture was uninvited. That's the strongest single indicator of poisoning.
3. **Bidirectional pattern** The two poisoned messages happened about 5 seconds apart, poisoning each side of the conversation. This is how a MITM is set up.
4. **Cross-check with other evidence** in a real investigation, you'd also want to check:
 - ARP cache change logs on the victim machines
 - Switch MAC-address-table entries (routers and managed switches keep track of MACs per port)
 - DNS or HTTP redirects from the victim
 - Unexpected TLS certificate warnings
5. ****Root cause**** — ARP itself has no built-in defence against forged replies. Detection can only come from watching the network externally.

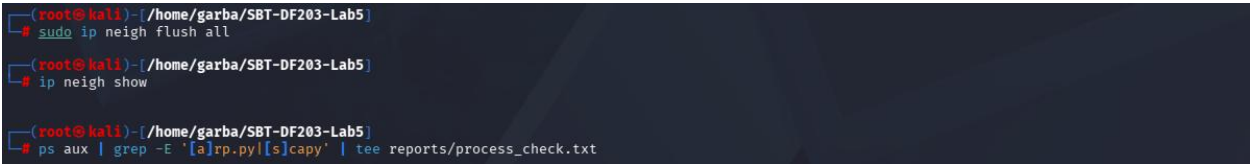
TASK 7: CLEANUP AND PREVENTION

Even though I didn't run any live attack, I documented the cleanup steps to show proper lab practice:

```
sudo ip neigh flush all

ip neigh show | tee reports/arp_table_restored.txt

ps aux | grep -E '[a]rp.py|[s]capy' | tee reports/process_check.txt
```



After the flush, my ARP cache was empty. The process check confirmed no `arp.py` or `scapy` scripts were left running on my machine.

How to Prevent ARP Poisoning

CONTROL	WHAT IT DOES (IN SIMPLE TERMS)
Dynamic ARP Inspection (DAI) with DHCP snooping	A managed switch that checks every ARP message against a trusted list. If a machine claims an IP that doesn't match the DHCP record, the switch throws the message away.
Port security	Limits how many MAC addresses can appear on a single physical port. Stops a machine from pretending to be many.
Static ARP entries for critical hosts	Manually code the correct IP-to-MAC mapping for your gateway and servers so they can't be overwritten.
Network segmentation (VLANs)	Split the network into small pieces so an attacker can only poison their own piece.
Endpoint ARP-monitoring tools(like arpwatch or XArp)	Software that watches the ARP cache and alerts you when something suspicious changes.
Encrypted protocols (HTTPS, VPN, SSH)	Even if an attacker gets in the middle, they can't read the contents — they're scrambled.
802.1X port authentication	Requires a password before a device can even connect to the network port.

CONCLUSIONS

The ARP analysis for Lab 5 was completed successfully. Here's what I learned:

1. What normal ARP looks like. A device broadcasts a question ("who has IP X?") and the owner replies directly with its MAC address. Clean, simple, and fast.
2. What ARP poisoning looks like. In the poisoned capture, two devices sent each other unsolicited ARP replies without being asked telling each other who owned which IP. That's the fingerprint of an attack.
3. Why this matters. ARP has no authentication. Any machine on the network can claim any IP-to-MAC mapping, and receiving machines will trust it. That makes poisoning easy and detecting it hard because from the victim's point of view, everything still works.
4. How to defend against it. The best protections are on the network side (Dynamic ARP Inspection, DHCP snooping, port security) and on the application side (use HTTPS, VPNs, SSH). You can't fix ARP itself, but you can put controls around it.
5. Why documentation matters. This lab reinforced forensic best practices:
 - Hash the evidence before you touch it
 - Keep original and working copies separate
 - Record every command and observation
 - Document anything you didn't do, and why
 - Clean up after yourself

STUDENT DECLARATION

I confirm that I completed this lab in an authorised environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.

Signed: Garba Ahmad Zubairu

Date: 18th Sept 2026